```
      state: State,
      command: Command): ReplyEffect[Event, State] = {            ❷
    Effect.none.thenReply(command.replyTo)(
      _ =>  RequestUnaccepted(
              s"[$command] is not allowed upon state [$state]"))
  }
}
```

❶ Creates an event, persists it, and replies to the caller

❷ Does not persist; only replies to the caller

There is nothing surprising in these methods. They persist an event and then respond, except `tell` and `invalid`: `tell` correspond to the `GetState` command, so there is nothing to persist when the actor is queried; and `invalid` is used to channel any command that does not match the actor's state, so again, there is nothing to persist. These two commands do not change the actor's state in content or in type.

12.1.3 The bet

The `bet` actor coordinates with the other two. When a client initializes a bet, the actor must verify that the market odds are still available (as they can change quickly) and that the wallet has enough money to cover the bet. If both verifications are successful, the bet is considered valid and remains open until settlement when the market closes.

Once a bet is opened, these two validations must occur in less than a few seconds. The actor schedules a check that, if not passed, rejects the bet. You'll see two different approaches to perform these checks.

On the one hand, you ask the market if the odds are still available or if the market has moved. You want to process this request in less